知错

面向编程初学者的 AI 实验诊断与补弱平台

作品定位

以真实实验结果为证据，连接基础测评、错误排查、AI 辅导、针对练习与复测验证。项目采用独立实现的 React + TypeScript 前端与 Node.js 后端，运行在本机。

参赛团队

学校：广东东软学院
团队：七场五点八二拉菲
成员：陈凯奕、邝鸿毅、龚栩敏
指导教师：杨雨倩
组别：B 组

文档状态

技术说明 · 2026-11-11。依据参赛者提供的赛道要求整理，描述当前实现与验证边界。公开代码仓库：github.com/loulan666/zhicuo。提交前请与最终录制视频核对。

01 / 项目概述

需求与应用价值

真实问题

初学者常遇到“能看懂答案、却不能独立完成代码”的落差。单纯给出完整答案可能跳过定位错误与验证理解的过程。知错将问题聚焦于基础概念、边界条件与实验反馈。

目标用户与流程

面向学习 Python、JavaScript、TypeScript、Java、C、C++、Go 的编程初学者。先完成阅读测评，查看先修知识，再编写函数并运行测试；根据失败证据获得排查方向与模型解释，修正后独立复测。

范围与非目标

当前为单人、本地学习空间，不提供多用户认证、公网执行服务或防作弊认证。课程覆盖入门概念，不声称覆盖所有主流语言的全部高级特性。没有开展真实班级对照研究，不报告虚构的学习提升比例。

02 / 系统架构

界面、证据、模型与执行分离

交互层

React + TypeScript 提供学习总览、课程图谱、基础测评、实验、资料库、AI 诊断、补弱复测、画像与环境检查。Coach 独立组件负责诊断请求、取消及历史展示。esbuild 构建浏览器资源。

服务层

Node.js HTTP 服务校验 Host 与 Origin，提供窄接口。学习数据保存在本地 JSON，写入采用队列与临时文件替换，避免并发覆盖。评测分数由后端计算，模型不能修改成绩。

执行层

Docker 在禁网、非 root、只读根目录且无主机目录挂载的容器中运行代码。每次执行限制 CPU、内存、进程、输出量及时间，结束后强制清理。Windows 使用 Docker Desktop 与 WSL2。

数据流

浏览器提交代码 → 后端包装函数与输入 → 容器运行 → 保存真实测试记录 → AI 只读工具读取证据 → 模型生成建议 → 学生练习与复测。API 密钥只保留在服务端 .env。

03 / AI 技术方案

有边界的证据诊断工作流

模型选型

使用 DeepSeek 的兼容 Chat Completions 接口。选择理由是中文教学解释能力与 Function Calling 接口适配，实际成本与响应速度仍需长期观测。本项目没有做模型横向基准评测，不能声称其优于所有其他模型。

Function Calling

最多四轮模型请求。系统先要求 read_evidence 读取当前语言最近的测评与实验，要求 get_practice_plan 获取按真实记录计算的补练候选；模型可调用 search_materials 检索课程资料。工具名称与参数经过白名单校验。

可解释输出

报告展示实际调用工具、返回证据数量、引用片段和耗时。提示词要求区分观察与推测，并按“证据、可能原因、下一步、如何验证”组织答案。展示的是工具执行轨迹，不是模型内部思维链。

优化与风险

真实测试发现时间戳和题目编号容易被误读，因此将提交日期改为明确字段，并提供 nextExerciseId。限制最近记录数、检索片段数、输出 token 和总请求轮次。模型仍可能判断错误，建议必须通过代码实验核对。

04 / 资料与学习证据

检索增强与补弱验证

资料检索

支持文本 PDF、DOCX、TXT 和 Markdown。文本按 700 字符切片，保留 100 字符重叠，按当前语言过滤并使用关键词评分返回前四个片段。向模型提供匹配原文，用户可展开来源。扫描 PDF 尚需先 OCR。

能力边界

当前是关键词检索增强，不是向量数据库方案。没有多模态理解或 MCP 协议服务。无资料匹配时明确不提供引用，不伪造来源。后续可在有评价集的基础上比较向量与混合检索。

测评与画像

七种语言共 126 道基础测评，每种语言 18 题、每维 3 题，覆盖阅读、推演与排错。雷达显示正确率和答对题数；支持分节草稿与错题课程跳转。旧版六题历史保留原分母。实验通过数、提示使用和复测状态分别展示，不把它们包装为智力或学习天赋。

补弱闭环

每种语言四类原实验与四组复测。失败用例产生零值、空输入、负数或运行错误等排查方向。先通过原题，再无提示通过后续复测才标记通过；后续失败更新状态。复测是同类任务换输入，不能证明已掌握全部概念。

05 / 工程与验证

可复现测试与安全边界

自动化验证

33 项单元和接口测试通过，覆盖新旧版本计分、存储并发、资料检索、来源校验、模型异常、工具白名单、语言隔离与复测时序；TypeScript 检查和前端构建通过。

真实运行验证

七种语言均通过正常输入与空输入编译执行；Python 与 Go 的 HTTP 提交、错误答案、语法错误及保存记录通过；四组 Python 复测通过容器验证。无限循环约 20 秒超时终止，容器完成清理。

浏览器与模型验证

在隔离测试数据目录完成桌面八页导航、基础测评提交、真实 Python 执行、DeepSeek 真实工具调用与报告保存；390 像素宽度总览无横向溢出，没有捕获到页面 JavaScript 异常。这不等于所有浏览器和设备均已验收。

本地安全边界

服务只绑定回环地址；限制输入、调用频率和模型超时；代码执行不挂载主机目录。工具只读，无任意命令和任意 URL 能力。取消请求中止等待；已产生的模型费用不保证退回。容器不是绝对安全保证，不应直接作为公网多租户服务。

06 / 安装与提交

评审复现与待完成事项

安装启动

准备 Node.js 22+、Docker Desktop 与 WSL2。执行 npm ci、npm run build；复制 .env.example 为 .env 并填写自己的模型密钥。启动 Docker，运行 scripts/setup-runtimes.ps1 下载镜像，然后 npm start。浏览器访问 http://127.0.0.1:3180/。

验证命令

npm test；npm run typecheck；node scripts/verify-runtimes.mjs；node scripts/verify-submissions.mjs。首次下载需要联网；模型问答持续需要网络，准备好的容器可在禁网环境运行学生代码。

提交材料

代码仓库须公开或给予评审访问权限，保留真实 Git 历史。演示视频应为 1080p 以上、MP4、5–8 分钟、普通话讲解。技术文档不超过 30 页。公网演示按提供要求为可选项。本地 Git 不能替代可访问的评审仓库。

项目实现与后续

知错将测评、容器执行、证据诊断与复测连接为学习流程，第三方依赖见 package-lock.json。开发使用 AI 辅助，团队应理解实现并如实答辩。下一步扩充题库、拆分页面组件、评估检索质量并开展学生试用。B 组和团队信息已确认。公开仓库为 github.com/loulan666/zhicuo。最终视频由团队自行录制，并在赛事系统完成材料提交。